

CS 486/686

Sequence Models, Attention, and Transformers

Yuntian Deng

Lecture 19

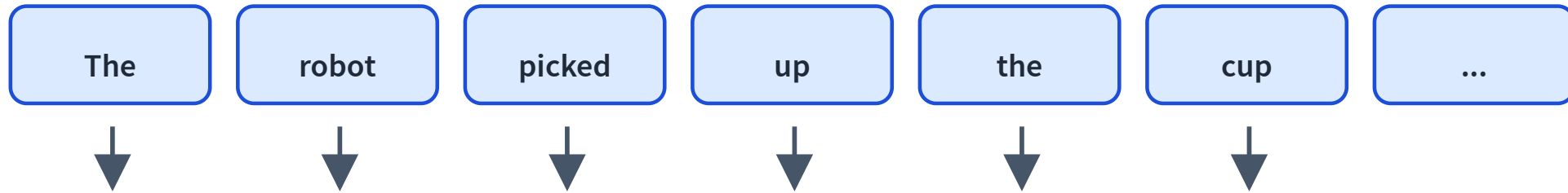
How embeddings talk to each other

Learning goals

- Model a sentence as a **sequence of embeddings**.
- Explain **RNNs** and why they struggle to scale.
- Explain **attention** as token-to-token retrieval.
- Assemble a **transformer block** and understand **causal masking**.

From embeddings to sequences

L18: each item can become a learned vector.



L19: how do these vectors interact?

A sentence is not a bag of words

dog bites man

man bites dog

Same words. Different order. Different meaning.

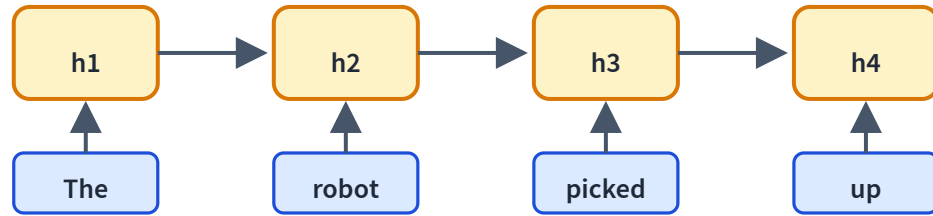
Running example

The robot picked up the cup because **it** was empty.

What does **it** refer to?

The answer depends on context, not the token alone.

The first idea: recurrent state



Read one token at a time.

$$h_t = f(h_{t-1}, x_t)$$

The hidden state summarizes the past.

RNN inference: update and pass forward

The

may encode:
determiner

robot

may encode: entity

picked

may encode: action

up

may encode: phrase

The hidden state is one dense vector, not a literal list;
these labels are intuition for what may accumulate.

Why RNNs are limited

Sequential

Step t waits for $t - 1$.

Bottleneck

Everything squeezes into one state.

Long paths

Distant tokens interact through many steps.

Good concept, hard scaling story.

Modern recurrence revisits the idea

Structured state updates can be faster and stabler:

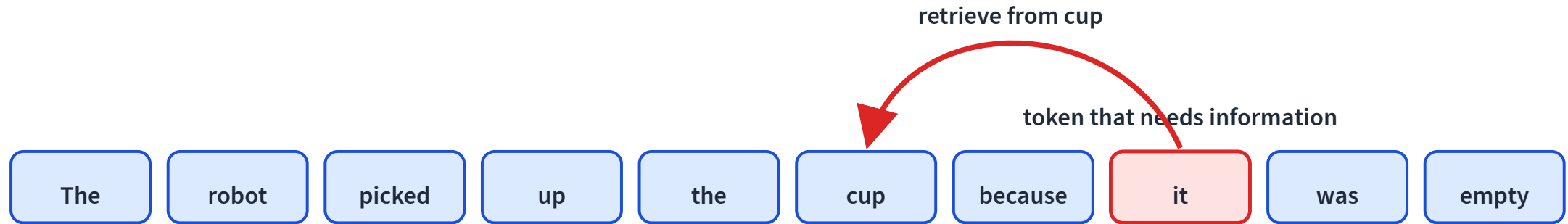
$$\text{state}_t = A \text{state}_{t-1} + Bx_t$$

A is a learned state update; B maps the new input into state.

Modern state-space models keep recurrence, but redesign it for parallel hardware.

Different idea: let tokens look directly

When processing **it**, which earlier word contains useful information?



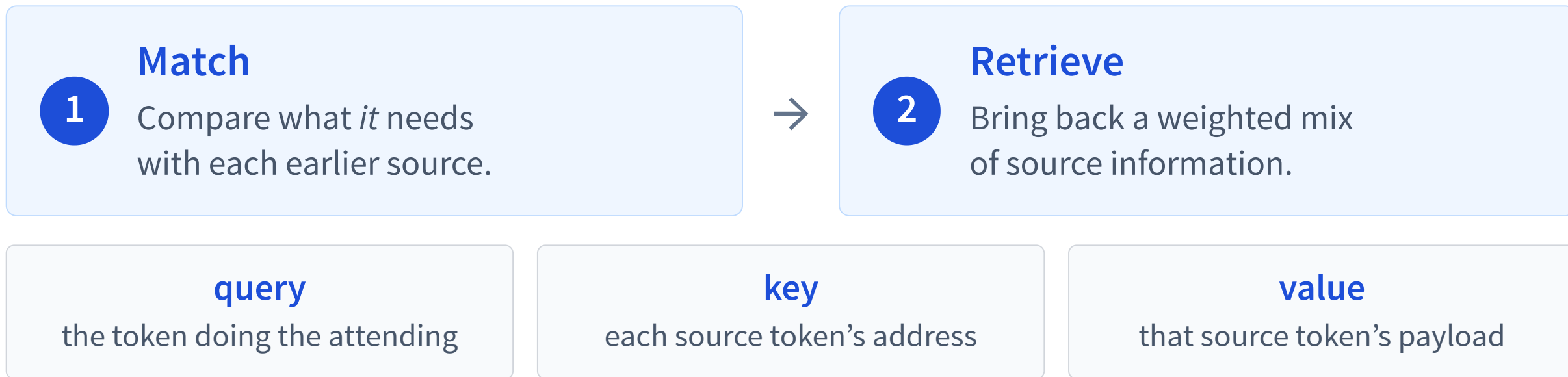
A decoder can look only at itself and earlier source tokens.

Reaching across the sequence

For an earlier source and a later query, the RNN path grows; attention stays one hop.

Interactive on the live slides: compare an earlier source with a later query. In an RNN, information crosses $n - 1$ sequential updates; causal attention lets the later query reach any earlier source in one hop, while computing $O(n^2)$ scores for the sequence.

Attention is content-based lookup



Keys decide where to look; values decide what comes back.

Queries, keys, values are learned projections

At the current layer, token i has hidden state $\mathbf{h}_i$:

need

$$\mathbf{q}_i = W_Q \mathbf{h}_i$$

query

address

$$\mathbf{k}_i = W_K \mathbf{h}_i$$

key

payload

$$\mathbf{v}_i = W_V \mathbf{h}_i$$

value

Same hidden state, three learned views. In deeper layers, $\mathbf{h}_i$ already contains context.

One query scores earlier keys

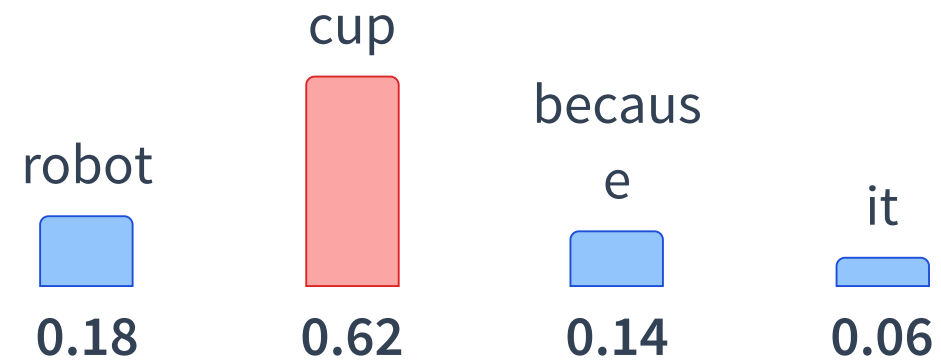
Current token: *it* supplies $\mathbf{q}_{it}$.

Allowed source $j: j \leq i$; it
supplies $\mathbf{k}_j$ now and $\mathbf{v}_j$ later.

$$\text{score}(it, j) = \mathbf{q}_{it}^\top \mathbf{k}_j$$

source key k_j	match score
robot	medium
cup	high
because	low
it (self)	low

Softmax turns scores into weights



The allowed source weights are nonnegative and sum to 1.

Use the weights to mix source values

$$\mathbf{z}_{it} = \sum_{j \leq i} \alpha_{it,j} \mathbf{v}_j$$

0.18 $\mathbf{v}_{\text{robot}}$

0.62 $\mathbf{v}_{\text{cup}}$

0.14 $\mathbf{v}_{\text{because}}$

0.06 $\mathbf{v}_{it}$



$\mathbf{z}_{it}$

new context vector

Because **cup** has the largest weight, its payload contributes most.

Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

$QK^\top$

all pairwise relevance scores

V

the information being mixed

d_k is the query/key width; dividing by $\sqrt{d_k}$ keeps large dot products from saturating softmax.

Scores → weights

Softmax turns raw scores into attention weights — scaling sharpens or flattens them.

Interactive on the live slides: drag the scores and the temperature and watch softmax turn them into a probability distribution. Low temperature (like large scores, or a small $\sqrt{d_k}$) sharpens attention onto one token; high temperature flattens it toward a uniform average.

Causal masking prevents peeking

		key position			
query		x	x	x	x
			x	x	x
				x	x
					x

A decoder predicts the next token from the prefix.

Query position i may use keys/values only from $j \leq i$.

$$\text{softmax}\left(\frac{QK^{\top} + M}{\sqrt{d_k}}\right) V$$

$$M_{ij} = 0 \text{ when } j \leq i$$

$$M_{ij} = -\infty \text{ when } j > i$$

Causal masking, interactively

Toggle the mask and pick a query word — watch the weights renormalize over allowed positions.

Interactive on the live slides: toggle the causal mask on and off and click a query word. With the mask on, every future cell is blocked and that token's attention renormalizes over positions $\leq t$ only; with it off, the token attends both directions. This one switch is what turns a transformer into a left-to-right language generator.

Every row asks where one query looks

	robot	picked	cup	because	it
robot		×	×	×	×
picked			×	×	×
cup				×	×
because					×
it					

Rows: query positions.

Columns: source keys/values.

Toy row: query **it** puts its largest weight on **cup**.

Gray cells are future positions blocked by the causal mask.

Real attention in a decoder

Actual Qwen3-0.6B causal attention — pick a head, then click a query word.

Interactive on the live slides: real attention from Qwen3-0.6B, the small decoder model dissected in L21. Future cells are causally masked. Pick a measured layer/head pattern and click a query word to inspect its attention row; one selected head gives high weight from **it** to **cup**, another emphasizes the previous token, and another spreads over the allowed prefix. Attention is inspectable, but is not automatically a faithful explanation.

Multi-head attention repeats the lookup

Each head learns different W_Q, W_K, W_V projections.

source link

high *it* $\rightarrow$ *cup*

local pattern

previous token

broad pattern

spread over prefix

Heads can emphasize different patterns in parallel; the real demo shows measured examples.

Transformers dropped recurrence—where did order go?

RNN

Order is built into sequential updates.

Transformer, layer 1

all token vectors processed in parallel

Content alone does not label first, second, third...

Fix the final query and shuffle the same previous vectors:
content-only first-layer attention is unchanged *by word*.

Position changes the attention scores

simple start: $\mathbf{h}_i^0 = \text{token}_i + \text{position}_i$

Now the projected queries and keys depend on both content and slot.

Modern decoders: Qwen/GPT-style models commonly rotate query/key dimensions with RoPE, encoding relative position directly in their match.

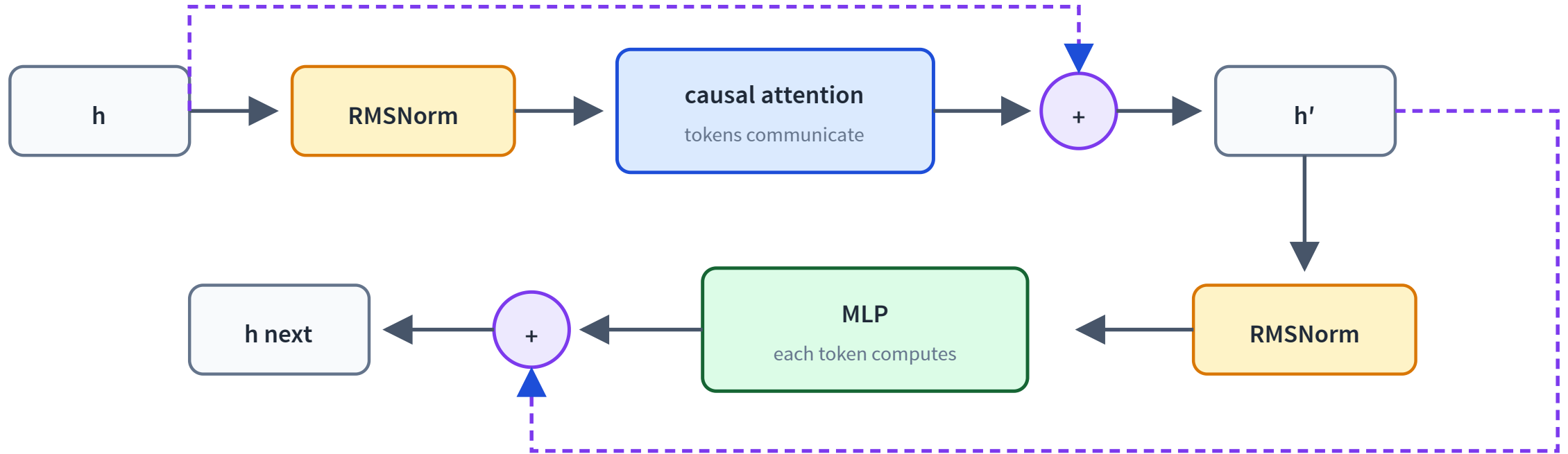
Mask: earlier vs future. **Position signal:** richer relative/absolute order.

Can the final token tell who did what?

At the final token **because**, the decoder is about to predict what comes next.

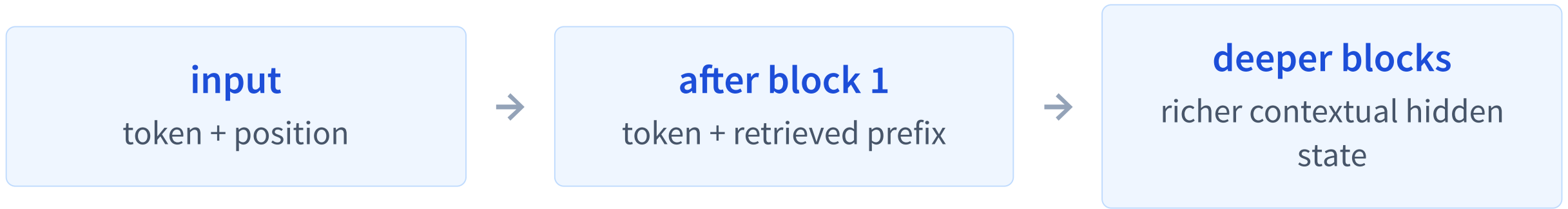
Interactive on the live slides: compare “dog bites man because” with “man bites dog because.” The final token **because** supplies the query vector and attends backward to keys/values from **dog**, **bites**, and **man**. Without positions, first-layer attention gives each word the same weight in both orders, so it builds the same context even though the meanings differ. Adding positions makes the contexts differ before next-token prediction. In deeper layers, source hidden states may also differ because they were built from different causal prefixes.

A decoder block: communicate, then compute



Residual paths preserve the old representation; normalization stabilizes scale. Exact variants differ—Qwen uses a pre-norm/RMSNorm-style block.

Keys and values become contextual



Shuffle previous words and their causal histories
change—so deeper-layer $\mathbf{k}_j$, $\mathbf{v}_j$ can change too.

This is why the first-layer “same set” result is useful but not the whole transformer story.

Two common attention masks

Bidirectional encoder

Every token may use both left and right context. BERT-style.

Causal decoder

Position i uses only $j \leq i$. Qwen/GPT-style.

Today's main path is the decoder: its mask enables next-token training and generation.

Why transformers took over

Parallel

All positions at once during training.

Long-range

Any allowed source is one attention hop away.

Scalable

Bigger models and data kept improving.

Now we have the architecture

Next we need the training task:

predict the next token at massive scale

L20: pretraining language models from first principles.

Recap

- ✓ RNNs summarize the past with a **hidden state**.
- ✓ Attention retrieves information directly between **tokens**.
- ✓ Queries, keys, and values implement learned retrieval.
- ✓ Transformer blocks stack **attention** and **feed-forward** computation.
- ✓ Causal masks enable autoregressive training and generation.